



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 3
Implement Noisy Gradient Descent algorithm
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Aim: To implement the Noisy Gradient Descent (NGD) algorithm to minimize a given function.

Theory:

Gradient Descent is an optimization algorithm used to minimize a function by iteratively moving in the direction of the negative gradient. Noisy Gradient Descent (NGD) is a variation of this method where random noise is added to the gradient to potentially escape local minima and enhance exploration in non-convex functions. The update rule for NGD is:

$$x_{t+1} = x_t - \eta(\nabla f(x_t) + \xi_t)$$

- x_t is the current parameter,
- η is the learning rate,
- $\nabla f(x_t)$ is the gradient of the function,
- ξ_t is the noise term (typically drawn from a normal distribution).

This method is particularly useful in deep learning and reinforcement learning, where noisy gradients help in better generalization and escaping sharp local minima.

Procedure/Algorithm:

1. Define the function $f(x)$ to be minimized.
2. Compute its gradient $\nabla f(x)$.
3. Initialize the starting point x_0 .
4. Set learning rate η and noise scale σ .
5. For a fixed number of iterations:
 - Compute the gradient $\nabla f(x_t)$
 - Generate random noise ξ_t
 - Update the parameter using the Noisy Gradient Descent update rule.
6. Store and visualize the convergence behavior.
7. Output the final minimized value and compare it with standard Gradient Descent.



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def function(x):
    return x**3 - 2*x + 2

def gradient(x):
    return 3*x**2 - 2

x_init = float(input("Enter the initial value of x: "))
learning_rate = float(input("Enter the learning rate ( $\eta$ ): "))
noise_scale = float(input("Enter the noise scale ( $\sigma$ ): "))
iterations = int(input("Enter the number of iterations: "))

x_ngd = x_init
x_gd = x_init
history_ngd = [x_ngd]
history_gd = [x_gd]

for i in range(iterations):
    noise = np.random.normal(0, noise_scale) # Generate random noise
    grad_ngd = gradient(x_ngd) # Compute gradient for NGD
    grad_gd = gradient(x_gd) # Compute gradient for GD

    x_ngd = x_ngd - learning_rate * (grad_ngd + noise) # Noisy GD update
    x_gd = x_gd - learning_rate * grad_gd # Standard GD update

    history_ngd.append(x_ngd)
    history_gd.append(x_gd)
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

```
if i < 10:
```

```
    print(f"Iteration {i+1}: x (NGD) = {x_ngd:.6f}, x (GD) = {x_gd:.6f}")
```

```
x_vals = np.linspace(-3, 3, 400)
```

```
y_vals = function(x_vals)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(x_vals, y_vals, label="f(x) = x3 - 2x + 2", color="black", linestyle="dotted")
```

```
plt.scatter(history_ngd, function(np.array(history_ngd)), color="red", marker="o", label="Noisy GD Path")
```

```
plt.scatter(history_gd, function(np.array(history_gd)), color="blue", marker="x", label="Standard GD Path")
```

```
plt.xlabel("x")
```

```
plt.ylabel("f(x)")
```

```
plt.title("Function and Gradient Descent Paths")
```

```
plt.legend()
```

```
plt.grid()
```

```
plt.show()
```

```
print(f"\nFinal optimized x (NGD) = {x_ngd:.6f}")
```

```
print(f"Function value at optimized x (NGD) = f({x_ngd:.6f}) = {function(x_ngd):.6f}")
```

```
print(f"\nFinal optimized x (GD) = {x_gd:.6f}")
```

```
print(f"Function value at optimized x (GD) = f({x_gd:.6f}) = {function(x_gd):.6f}")
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Output:

Enter the initial value of x: 3

Enter the learning rate (η): 0.1

Enter the noise scale (σ): 0.9

Enter the number of iterations: 5

Iteration 1: x (NGD) = 0.550478, x (GD) = 0.500000

Iteration 2: x (NGD) = 0.717909, x (GD) = 0.625000

Iteration 3: x (NGD) = 0.829382, x (GD) = 0.707812

Iteration 4: x (NGD) = 0.823068, x (GD) = 0.757513

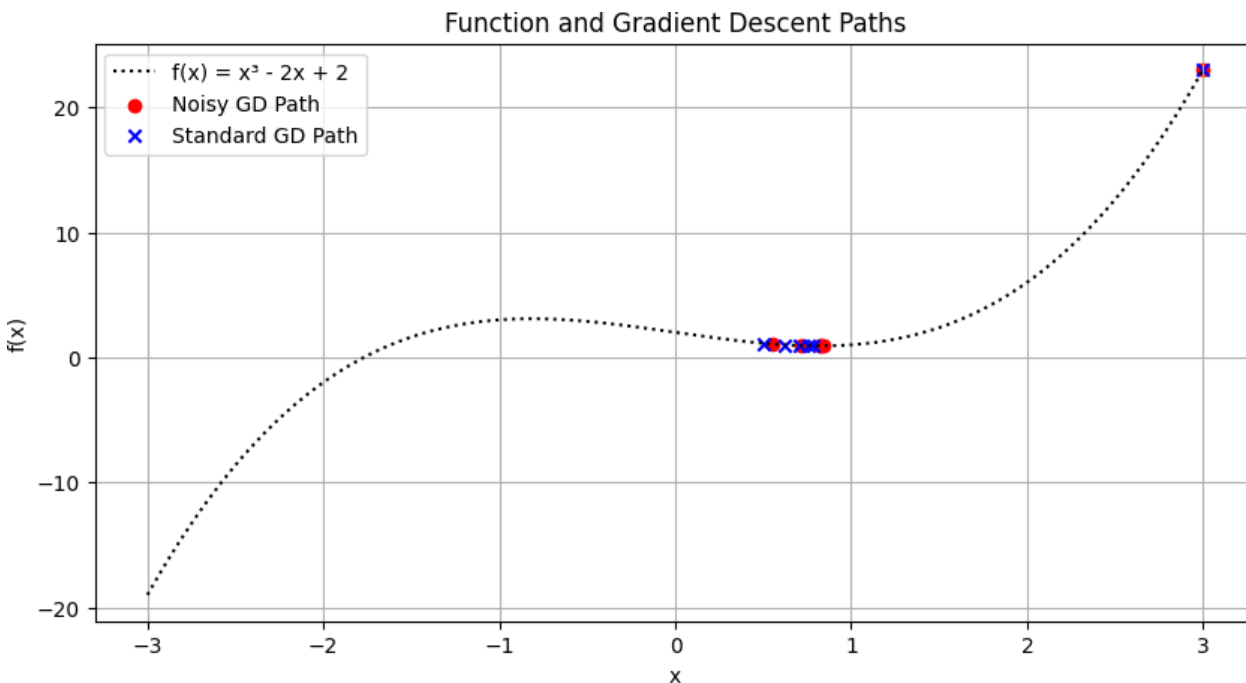
Iteration 5: x (NGD) = 0.838793, x (GD) = 0.785365

Final optimized x (NGD) = 0.838793

Function value at optimized x (NGD) = $f(0.838793) = 0.912567$

Final optimized x (GD) = 0.785365

Function value at optimized x (GD) = $f(0.785365) = 0.913682$





Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Conclusion: We learned how Noisy Gradient Descent introduces randomness to exploration, while Standard Gradient Descent follows a steady path to optimization.

Answer the following questions:

Q1: Why is noise added to the gradient in Noisy Gradient Descent?

Noise is added to the gradient in Noisy Gradient Descent (NGD) to help the algorithm explore different paths and avoid getting stuck in local minima. In standard Gradient Descent, the algorithm follows a fixed path based on the gradient, which might lead it to a local minimum instead of the best global solution. By adding noise, NGD introduces some randomness in the updates, allowing it to escape small dips (local minima) and explore better solutions. This technique is especially useful in complex functions where multiple peaks and valleys exist, making it harder for standard Gradient Descent to find the best answer.

Q2: How does Noisy Gradient Descent compare to standard Gradient Descent?

Noisy Gradient Descent (NGD) and Standard Gradient Descent (GD) both aim to find the lowest point of a function, but they work in different ways. Standard GD follows a smooth and steady path, always moving in the direction that decreases the function's value the most. However, it can sometimes get stuck in small dips (local minima) and may not reach the best possible solution (global minimum). In contrast, NGD adds a small amount of random noise to the updates, which helps it jump out of small dips and explore different paths. This makes NGD useful for complex functions with many ups and downs, but it can also make the path to the minimum less predictable.